

Лабораторная 9. Веб + API KnowBase

Цель работы

Научиться находить технические ошибки веб-приложения через вкладки Network и Console, переносить обнаруженные API-запросы в Burio, проверять позитивные и негативные сценарии для статей и комментариев, а также безопасно подтверждать нарушение объектной авторизации IDOR.

Что должно получиться

- исследованы основные действия со статьями и комментариями во вкладках Network и Console
- описаны найденные технические ошибки или доказательно зафиксировано их отсутствие
- в Burio собрана коллекция запросов с окружением и переменными
- для каждого эндпоинта из выданного контракта выполнен минимум один позитивный и один негативный сценарий
- безопасно проверена объектная авторизация для комментариев и оформлен баг-репорт на подтверждённый IDOR
- сформулирован вывод о согласованности веб-интерфейса и API

Список ссылок

Здесь добавить ссылку на учебный стенд KnowBase.

Здесь добавить ссылку на Swagger для API статей и комментариев.

Здесь добавить готовую коллекцию Burio или ссылку на архив с коллекцией, если она подготовлена.

Здесь добавить две учебные учётные записи одинаковой роли: пользователь А и пользователь В.

Здесь добавить инструкцию по получению токена или готовые временные токены без публикации в общем доступе.

Здесь добавить идентификатор тестовой статьи либо правило создания отдельной статьи для работы.

До выдачи работы. Лабораторную нельзя выполнять без отдельного учебного стенда, API-контракта и двух контролируемых учётных записей. Если хотя бы одного элемента нет, студент отмечает соответствующую часть как заблокированную и не подбирает данные самостоятельно.

Правила безопасной работы

- работайте только с адресом, который выдал преподаватель, и не переносите запросы на другие системы
- используйте только две выданные учебные учётные записи и созданные вами тестовые данные
- не перебирайте последовательные ID, не запускайте сканеры, циклы, нагрузочные тесты и массовые запросы
- для IDOR используйте ID комментария, который пользователь А создал специально для этой работы

- не проверяйте доступ к комментариям других студентов и реальных пользователей
- не вставляйте токены, cookie и персональные данные в отчёт и на снимки экрана
- после подтверждения дефекта прекратите повторные попытки и сообщите преподавателю

Важно. Цель работы состоит в подтверждении заранее предусмотренного учебного дефекта, а не в поиске новых способов обхода защиты. Одна доказательная проверка достаточна.

Установка Bruno

Bruno - бесплатный API-клиент с открытым исходным кодом. Для лабораторной достаточно настольной версии. Регистрация и платная подписка не нужны.

1. Откройте официальную страницу <https://www.usebruno.com/downloads>.
2. Выберите установщик для Windows, macOS или Linux. На учебном компьютере используйте вариант, разрешённый преподавателем или администратором.
3. Установите приложение стандартным способом и запустите его.
4. Создайте пустую коллекцию KnowBase Lab 9
5. Если установка запрещена политикой компьютера, остановитесь и запросите разрешённый вариант. Не скачивайте неофициальные сборки.

Альтернатива для Windows. Если пакетный менеджер Winget уже установлен и его использование разрешено, Bruno можно установить командой `winget install Bruno.Bruno`. Командная строка не обязательна для выполнения лабораторной.

Подготовка

1. Откройте Swagger и выпишите все эндпоинты статей и комментариев, которые входят в выданный объём.
2. Войдите в KnowBase под пользователем А. Не закрывайте Swagger и не удаляйте тестовые данные до завершения работы.
3. Откройте DevTools клавишей F12 или сочетанием Ctrl+Shift+I. На macOS используйте Cmd+Option+I.
4. Во вкладке Network включите запись, установите фильтр Fetch/XHR, очистите список и включите Preserve log, если переходы между страницами очищают журнал.
5. Во вкладке Console очистите прежние сообщения. Зафиксируйте версию браузера, адрес стенда, дату и время начала проверки.

Часть А. Исследование KnowBase через DevTools

Выполните действия последовательно. После каждого действия найдите связанный запрос во вкладке Network. Смотрите Headers, Payload, Response и Timing. Одновременно проверяйте, появились ли новые ошибки в Console.

1	Открыть список статей	Метод и URL, статус, структура списка, ошибки загрузки и сообщения Console.
2	Открыть одну статью	ID статьи в URL или параметрах, статус, содержимое ответа, повторные или лишние запросы.
3	Создать тестовый комментарий А	Метод, Payload, статус создания, ID нового комментария, совпадение ответа с интерфейсом.
4	Обновить страницу статьи	Сохранился ли комментарий, какой запрос загрузил комментарии, нет ли 4xx/5xx и ошибок JavaScript.
5	Изменить комментарий А, если функция есть	Метод PATCH или PUT по контракту, переданные поля, статус и обновление интерфейса.
6	Удалить отдельный пробный комментарий, если функция есть	Метод DELETE, статус, отсутствие удалённой записи после повторной загрузки.

Как распознать ошибку

- запрос завершился статусом 4xx или 5xx в обычном позитивном пользовательском сценарии
- интерфейс показывает успех, но API вернул ошибку, либо наоборот
- после одного действия отправляются дублирующие изменяющие запросы
- ответ API не соответствует Swagger или содержит обязательные поля неправильного типа
- Console показывает красное сообщение JavaScript, необработанное исключение, ошибку CORS или отказ загрузки ресурса
- в URL, Payload, Response или Console раскрываются токены, пароли либо другие данные, которые не должны быть видны пользователю

Не выдумывайте дефект. Предупреждение Console не всегда означает пользовательскую ошибку, а статус 4xx может быть ожидаемым результатом негативной проверки. Сопоставьте наблюдение со сценарием, контрактом и фактическим влиянием.

Таблица наблюдений DevTools

Заполните минимум пять строк. Для ошибок приложите снимок экрана без токенов и cookie.

Часть В. Проверка API в Bruno

4.1. Создание окружения

Создайте окружение lab9 и добавьте только те переменные, которые нужны по выданному контракту:

baseUrl	Базовый адрес учебного API без завершающего слэша.
tokenA	Временный токен пользователя А.
tokenB	Временный токен пользователя В.
articleId	ID разрешённой тестовой статьи.
commentAId	ID комментария, созданного пользователем А.

4.2. Перенос запросов

1. Сопоставьте запросы из Network с описанием Swagger. DevTools показывает фактический вызов интерфейса, а Swagger задаёт контракт.
2. Создайте в коллекции папки Articles и Comments.

3. Для каждого эндпоинта задайте метод, URL через {{baseUrl}}, заголовки, параметры и JSON-тело.
4. Сохраните позитивный запрос с понятным названием, например Articles - get by valid ID.
5. Создайте отдельную копию для негативного сценария и назовите условие, например Articles - get by nonexistent ID.

4.3. Обязательный объём

Проверьте все эндпоинты статей и комментариев, перечисленные в выданном Swagger и отмеченные преподавателем как входящие в лабораторную. Для каждого выполните минимум два сценария.

Позитивный	Передать корректные данные и необходимые права.	Успешный статус по контракту, корректное тело и изменение состояния.
Негативный	Изменить одно значимое условие: убрать авторизацию, передать неверный ID или невалидное тело.	Контролируемый статус 4xx, понятный ответ, состояние данных не изменилось.

Правило одного изменения. В негативном сценарии меняйте одно условие. Если одновременно убрать токен, сломать JSON и заменить ID, невозможно понять причину результата.

Варианты негативных сценариев

- запрос без токена или с недействительным учебным токеном
- несуществующий ID в согласованном формате, который выдал преподаватель
- обязательное поле отсутствует
- пустая строка или значение длиннее документированного ограничения
- неверный тип данных, например строка вместо числового ID
- попытка изменить неизменяемое поле, если это запрещено контрактом

Матрица API-проверок

Добавьте строки так, чтобы в таблице присутствовали все эндпоинты из выданного объёма.

Позитивный				
Негативный				
Позитивный				

Негативный			
Позитивный			
Негативный			
Позитивный			
Негативный			

Часть С. Безопасная проверка IDOR в комментариях

IDOR возникает, когда сервер принимает идентификатор объекта от клиента, но не проверяет, имеет ли текущий пользователь право работать именно с этим объектом. Скрытая кнопка в интерфейсе не является защитой. Решение должен принимать сервер для каждого запроса.

Предусловие. Пользователи А и В имеют одинаковую обычную роль. Пользователь А создал специальный комментарий с текстом LAB9-A-<инициалы>. Его ID получен из ответа API, а не найден перебором.

Алгоритм проверки

1. Под пользователем А создайте специальный комментарий и сохраните его ID в commentAId.
2. Убедитесь, что пользователь А может изменить свой комментарий разрешённым методом из Swagger.
3. Скопируйте этот запрос и замените только токен А на tokenB. ID, URL и тело запроса оставьте прежними.
4. Отправьте один запрос на изменение комментария А от имени пользователя В. Используйте безвредный маркер LAB9-IDOR-B.
5. Ожидайте отказ 403 или скрытие существования объекта ответом 404, если именно такое правило задано контрактом.
6. Если сервер вернул успешный статус, проверьте под пользователем А, действительно ли текст изменился. Совпадение успешного статуса и изменения подтверждает IDOR.
7. Не повторяйте запрос. Сохраните обезличенные доказательства и сообщите преподавателю.
8. Восстановите или удалите специальный комментарий под пользователем А, если это предусмотрено инструкцией стенда.

Если изменение не поддерживается. Используйте другой изменяющий эндпоинт комментария, который указан преподавателем. Не переходите к удалению и не выбирайте чужой объект самостоятельно.

Как оценить результат

403 или предусмотренный контрактом 404, комментарий не изменён	IDOR не подтверждён для этого сценария.
401	Запрос не авторизован. Сначала проверьте корректность tokenB, затем повторите не более одного раза.
200 / 204, но комментарий не изменён	Результат неоднозначен. Проверьте контракт и оформите наблюдение, но не объявляйте IDOR без подтверждения.
200 / 204 и комментарий A изменён пользователем B	IDOR подтверждён. Оформите баг-репорт высокой серьёзности по учебной шкале.
5xx	Серверная ошибка. Это дефект, но сама по себе она не доказывает IDOR.

Оформление дефектов

Оформите отдельный баг-репорт на каждый уникальный дефект Network, Console или API. Одинаковое проявление в нескольких запросах можно объединить, если у него одна причина и понятен объём влияния.

Заголовок	Где, при каком условии и что произошло.
Окружение	Адрес учебного стенда, дата и время, браузер, версия сборки, Bruno.
Предусловия	Учётная запись, статья, специальный комментарий и необходимые права.
Шаги	Короткая последовательность с методом и путём. Токены не указывать.
Фактический результат	Статус, безопасный фрагмент ответа и фактическое изменение данных.

Ожидаемый результат	Поведение по Swagger, требованиям или правилу авторизации.
Доказательства	Снимок Network, Console или Bruno без секретных данных.
Серьёзность	Высокая, средняя или низкая с коротким обоснованием влияния.

Структура отчёта

1. Титульный лист с названием работы, ФИО, группой и датой.
2. Окружение и выданный объём: адрес стенда, версия или дата сборки, браузер, перечень эндпоинтов без токенов.
3. Заполненная таблица наблюдений DevTools и снимки найденных ошибок.
4. Матрица API-проверок: все эндпоинты, позитивные и негативные сценарии, ожидаемые и фактические результаты.
5. Результат IDOR-проверки с точными шагами и обезличенными доказательствами.
6. Баг-репорты на все подтверждённые уникальные дефекты.
7. Вывод объёмом 5-8 предложений: что работает, какие риски найдены, согласованы ли UI и API, что было заблокировано.

Отчёт сдаётся в формате docx

Критерии оценивания

DevTools: проверены основные действия, заполнено не менее пяти наблюдений, ошибки Network и Console оценены доказательно.	1 балл
Bruno: покрыты все эндпоинты из выданного объёма, для каждого есть позитивный и негативный сценарий.	2 балла
IDOR: проверка выполнена безопасно на двух учебных аккаунтах, результат интерпретирован корректно.	1 балл
Оформлены необходимые баг-репорты, доказательства обезличены, вывод опирается на результаты.	1 балл
Итого	5 баллов